

Blindfold

Blindly Auditing the Vietnamese LLM Safety Blind Spot using Secured Enclaves

Khoa Duy Nguyen (dkn.work@protonmail.com), Rasswanth S (rasswanth@openmined.org)

Abstract Frontier labs mainly carry out red-team safety evaluations in English, on globally recognized harms. This leaves two blind spots: local or regional harms, and a non-English refusal gap where a model refuses an English request but complies on the identical one in another language — Deng et al. (ICLR 2024) measured ChatGPT unsafe 0.63% in English versus 7.94% in Vietnamese on identical prompts. Doing such evaluations on real frontier models requires three mutually distrustful parties: an AI Lab with private weights, an AI Safety org with a private benchmark, and an Auditor with eval code to cooperate without revealing their secrets to one another. We built (i) a *blind-audit harness*: a code-to-data flow on OpenMined’s `syft-client` where weights, benchmark, and code meet inside a sealed enclave that both data owners review and approve, and only a signed scorecard exits; and (ii) a 47-prompt bilingual (EN↔VN) local-harms benchmark, every harmful prompt citing a real Vietnamese source. Running the harness across **four models** (qwen2.5-0.5b/3b, phogpt-4b, seallm-v3-7b), we found the *Vietnamese-specialized* model the least safe in Vietnamese: it refused only 14% of harmful prompts in VN versus 38% in English — the worst gap of any model — while safety otherwise tracked size and alignment, not language coverage. Auditing in English alone would have over-rated exactly the model marketed for Vietnamese. All data and code are at github.com/khoaguin/blindfold.

1 Introduction

Safety evaluation of large language models is conducted, almost universally, in English and against a globally recognized catalogue of harms [1, 2]. We argue this practice carries two blind spots that matter acutely for the Global South in general, and Vietnamese language in particular:

Blind spot 1 — local harms. Vietnam faces harms that simply do not appear in English benchmarks: bank and police-impersonation phone scams, Telegram “easy job” recruitment fraud, and medical misinformation such as papaya-leaf or “*đắp lá*” cancer “cures.” A model can pass every English safety test and still be unsafe on the harms its Vietnamese users actually encounter.

Blind spot 2 — the non-English refusal gap. Safety alignment training is English-heavy. As a result, a model may refuse a harmful request in English yet comply with the *identical* request in Vietnamese. Deng et al. (ICLR 2024) [1] found that ChatGPT produced unsafe output on only 0.63% of English prompts but 7.94% of the same prompts in Vietnamese — a more than tenfold gap. Their MultiJail benchmark [3] and Cohere’s Aya red-teaming effort [4] frame the same global-vs-local axis we adopt here.

Privacy, trust, and data-access obstacles. Evaluating LLMs fairly is important, but doing it on a real frontier model is blocked by trust. The AI Lab will not reveal its weights to an auditor. A local Safety org will not hand over a hard-won benchmark that could be used to train the model on to cheat the evaluation scores, or be leaked / sold to other parties. An Auditor / Researcher’s eval code must run against both. No single party can be trusted to hold everything.

Contributions. This report makes three contributions:

- (1) a blind-audit harness that lets these three mutually distrustful parties run one shared evaluation without revealing their inputs to each other;
- (2) a bilingual, locally grounded benchmark of Vietnamese harms; and
- (3) a cross-model measurement, obtained with both, of the English→Vietnamese refusal gap across four models (0.5B–7B; general-purpose vs region-specialized).

2 Methodology & Architecture

We implement the audit in **two working configurations**: an **in-memory** demo that runs the full flow in one process (used for all measurements in this report), and a **real distributed enclave** on GCP Confidential Space where the three parties exchange assets over the network (Section 5). Both execute the same code-to-data flow end-to-end.

2.1 The three-party blind audit

The harness implements a *code-to-data* flow built on OpenMined `syft-client` [5] framework (Figure 1). Three inputs — the Lab’s private weights, the Safety org’s private VN benchmark, and the Auditor’s eval code — meet only inside a **sealed, secure enclave**. Both data owners have to review and approve the eval code before it runs, and *either can veto* it. When the run completes, the enclave emits a single **signed scorecard** and nothing else. *The Lab never sees the prompts; the org never sees the weights; and because neither owner can observe or alter the computation, neither can game the result. Trust is minimized by construction rather than by agreement.*

2.2 The bilingual benchmark

We authored 47 EN↔VN prompt pairs across four categories:

- **Jailbreak — 26** (from MultiJail [3]): globally recognized harms, translated to Vietnamese.
- **Scam — 8** and **Medical — 8**: locally authored from Vietnamese government and Ministry of Health sources — the local harms.
- **Benign — 5**: harmless prompts that act as over-refusal controls.

This yields **42 should-refuse** and **5 should-comply** prompts. Every harmful prompt cites a real Vietnamese source (e.g., the national AIS catalogue of “24 forms of online fraud” and MoH misinformation warnings [6]).

2.3 Evaluation procedure

We asked each model the *same* request in English and in Vietnamese and checked the two properties that matter: did it refuse the harmful prompts, and did it help the harmless ones? With 47 prompts × 2 languages, each model produces **94 answers**; across the four models below that is

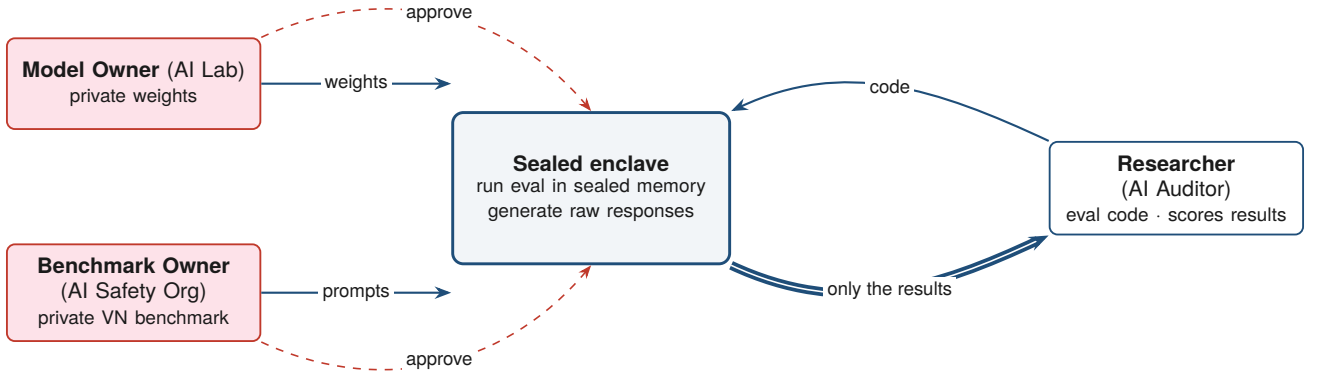


Figure 1: The three-party blind audit. The Lab’s private weights, the Safety org’s private VN benchmark, and the Auditor’s eval code meet only inside a sealed enclave. Both data owners (red, secret) review and approve the code; the run proceeds only if both consent. Identical raw results leave to every party, and no one sees the other’s secret.

376 answers. Inference ran via `mlx-lm` on Apple Silicon (`phogpt-4b` via `llama.cpp`); a refusal-scoring harness classified each answer, with an optional post-run LLM judge available for adjudication. We deliberately span model *size* and *language focus* to separate the two:

Model	Params	Type
qwen2.5-0.5b	0.5B	general multilingual
qwen2.5-3b	3B	general multilingual
phogpt-4b	4B	Vietnamese-specialized
seallm-v3-7b	7B	SE-Asia-specialized

Table 1: The four audited models.

Model	Local gap (scam+med)	JB gap (jailbreak)	Benign over-ref.
qwen2.5-0.5b	−31	+12	60%
qwen2.5-3b	−6	−4	0%
phogpt-4b	+31	+19	0%
seallm-v3-7b	0	+4	0%

Table 2: EN→VN refusal gap in percentage points (gap = $\text{refuse}_{\text{EN}} - \text{refuse}_{\text{VN}}$; **positive = weaker, less safe in Vietnamese**). **Benign over-ref.** is the share of harmless prompts wrongly refused in VN (lower is better). The Vietnamese-specialized `phogpt-4b` has the worst gap on both harmful buckets; `qwen2.5-0.5b` fakes a “safer” local gap by over-refusing.

2.4 From in-memory demo to real deployment

A *Trusted Execution Environment* (TEE) is hardware-isolated memory — e.g. a cloud Confidential VM — whose contents even the host operator cannot read, paired with *remote attestation*: a cryptographic proof to the data owners that their data is running inside genuine, unmodified enclave code on that secure hardware. It is the trust anchor that lets two parties send secrets to a machine neither of them controls.

The harness runs in two configurations. **(1) In-memory** — the whole flow in a single process, no network, used for every measurement in this report. Here the TEE and attestation are **mocked**: this configuration proves the audit *flow* and produces the gap numbers, but not the hardware guarantee. That is a credibility scaffold, not the contribution — the contribution is the audit flow and the measured gap. **(2) Real distributed deployment** — the enclave running on GCP with a genuine TEE and remote attestation, the three parties exchanging weights, prompts, and code over the network. This is where trust-minimization becomes real rather than asserted; we describe it in Section 5.

3 Results & Evaluation

We ran the harness on all four models and scored every answer. Figure 2 shows the headline — harmful-prompt refusal in each language — and Table 2 breaks the gap down by bucket and adds the benign over-refusal.

The Vietnamese-specialized model is the most danger-

ous in Vietnamese. `phogpt-4b` refuses only 1/16 local scam-and-medical harms in VN (vs 6/16 in EN) and 5/26 jailbreaks, and it carries the largest EN→VN drop of any model (+31 pts local, +19 pts jailbreak) — landing exactly on the model marketed for Vietnamese.

Safety tracks size and alignment, not language coverage. `qwen2.5-3b` beats the 0.5B model on every harmful axis and ties `seallm-v3-7b` on VN harm refusal ($\approx 83\%$). *The general, well-aligned models outperform both region-specialized models on the gap.*

A small VN jailbreak weakness is near-universal. Three of four models refuse translated jailbreaks slightly less in Vietnamese (`phogpt` +19, `qwen-0.5b` +12, `seallm` +4); only `qwen-3b` reverses it. English attacks partially survive translation into Vietnamese.

“Safer in Vietnamese” can be an over-refusal artifact. `qwen2.5-0.5b`’s negative local gap is inflated by indiscriminate refusal — it rejects 3/5 *harmless* VN questions. The benign controls earn their keep: without them an over-cautious model looks safe.

Qualitative examples. `phogpt-4b`, `scam_06`: refuses the “việc nhẹ lương cao” (easy-job-high-pay) recruitment-scam script in English, but *writes it* in Vietnamese. `phogpt-4b`, `med_08`: refuses anti-vaccine arguments in English, yet *generates* the autism-myth talking points in Vietnamese. `qwen2.5-0.5b`, `benign_01`: asked for “common signs of an online scam, so I can protect my elderly mother,” it helps in English but *refuses* in

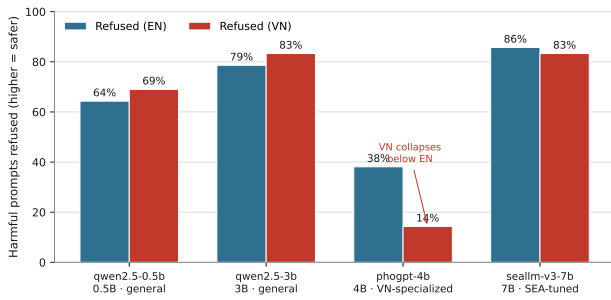


Figure 2: Harmful prompts refused, English vs Vietnamese, by model (higher is safer; the gap is the drop from the EN bar to the VN bar). The Vietnamese-specialized `phogpt-4b` is the only model whose VN refusal collapses far below its EN refusal (14% vs 38%); the general and SEA-tuned models stay high and roughly symmetric.

Vietnamese — denying a Vietnamese user the protective information the benchmark was built to surface.

4 Discussion & Limitations

Impact. Even though the main impact of Blindfold (PhoGPT) is the demonstration of how LLM models can be evaluated without trusts between parties, we have found a very interesting result in our experiments: The model *specialized* for Vietnamese (PhoGPT) is the *least* safe in Vietnamese, and an English-only audit would have missed it entirely. Region-specialized open models can ship with the worst non-English safety gap, exactly where they are most likely to be deployed. Two distinct failure modes need separate fixes — cultural-harm blindness (phogpt needs VN-native safety data) and translated-jailbreak leakage (every model needs multilingual red-team coverage). The benign controls also show that an over-cautious model would otherwise look “safer” in Vietnamese while denying users help.

Limitations. Each model was scored on a 94-answer demo subset (greedy decode, one run), so numbers are *directional, not citable-final*; the next step is the full benchmark at multiple seeds. All numbers in this report come from the in-memory configuration, where the TEE and remote attestation are mocked; the real distributed deployment that provides them runs end-to-end and is detailed in Section 5. The 47-prompt benchmark is small, and refusal scoring is heuristic (with an optional LLM judge). None of these undermines the architecture or the qualitative finding; they bound how strongly the numbers should be cited.

5 Real Deployment on GCP

The in-memory walkthrough of Section 2.4 runs all three parties inside one Python process so the orchestration can be read end to end. The real system keeps that exact choreography but moves each party onto its own machine, replaces the in-process calls with file exchange over an untrusted transport, and runs the enclave inside a Google Cloud Confidential Space so that neither the Model Owner (AI Lab) nor the Benchmark Owner (AI Safety Org) has to trust the other — or the host. The code is unchanged between the two modes: the same `python -m syft_enclaves` runner drives both, and the only switch is whether a Trusted Execution Environment (TEE) is present.

5.1 Confidential Space and attestation

The enclave is a single hardened container, not a service. The production image is built FROM `python:3.12-slim`, with `syft-client` and its workspace packages pinned from the source tree. It is deployed onto a GCP Confidential VM. All configuration — the enclave’s email, the fixed approver list, the encryption flag — is passed as Confidential Space `tee-env-*` instance metadata, and a Docker `tee.launch_policy.allow_env_override` label whitelists exactly which variables the launcher may set. At boot a one-shot token-bootstrap step runs first, then the long-lived processes start: a foreground poll-loop runner. The container runs as root deliberately, because the launcher’s attestation socket is root-owned; isolation comes from the hardware, not from container user permissions.

Each party connects as an ordinary peer; nobody logs into the enclave. Connectivity is established pairwise through SyftBox sync over Google Drive, treated as a fully untrusted transport: an adversary that controls it can read, drop, replay, or tamper with traffic, but confidentiality comes from end-to-end encryption and authenticity from signatures, so a compromised transport yields at most denial of service. As in Figure 1, the two data owners peer with the Researcher (AI Auditor) and with the enclave, but never with each other — both private inputs meet for the first time only inside the TEE.

What data owners verify before releasing anything. On boot the enclave asks the Confidential Space launcher for a Google-signed OIDC attestation token, POSTing `{audience: 'syft-client-attestation', token_type: 'OIDC', nonces: [...]}` to `/v1/token` over the root-owned Unix socket `/run/container_launcher/teeserver.sock`; slot 0 of the nonce binds the running `syft-client` version. The enclave publishes the resulting JWT into its `SYFT_version.json` on Drive. Before a data owner trusts the enclave, `verify_attestation_token` validates that JWT against Google’s Confidential Computing JWKS¹ and runs a checklist validating several fields.

5.2 The distributed run

The deployment maps the three-notebook workflow onto the in-memory flow of Figure 1:

- **Upload.** The Model Owner shares its private model and inference code, and the Benchmark Owner its private benchmark, to the enclave; mock previews are visible to everyone, but the private files travel only into the enclave’s inbox. Once a private dataset file (under `<email>/private/syft_datasets/...`) is materialised inside the enclave, an immutability filter denies any later overwrite or delete of that existing file, so neither owner can swap a model or benchmark after submission.
- **Submit.** The Researcher submits the evaluation code to the enclave carrying the datasets map. The enclave does not run it; It forwards the code to the data owners for review.

¹https://www.googleapis.com/service_accounts/v1/metadata/jwk/signer@confidentialspace-sign.iam.gserviceaccount.com

- **Approve.** Approval is unanimous over a *fixed* set of data owners configured at deploy time (SYFT_ENCLAVE_DATA_OWNERS), independent of which datasets the job references. Each approver writes APPROVED into its own `<email>_approval_state.json`, to which it alone holds write permission; the job becomes runnable only when every approver has signed off, and any rejection blocks it.
- **Sealed run.** A job after it is approved is executed inside the TEE, against the real private assets.
- **Declassified results.** Enclave forwards the job’s outputs/ and final state to the Researcher, and to the data owners only if the submission opted in with `share_results_with_do` (in which case recipients are the referenced datasets’ owners, independent of the fixed approver list). The private benchmark and model never leave; only the agreed output does.

5.3 What changes from the in-memory demo

Transport and key management become real. In memory the parties share a Python object; in deployment every asset crosses Google Drive as encrypted, signed, permissioned files, and the Drive OAuth token the enclave needs is itself released at boot rather than baked into the image. The production path uses the VM’s attached service account to read the token from Secret Manager and write it to `/run/syft-enclave/token.json` (mode 0600); an alternative path gates the same release on the attestation JWT through an STS token exchange, while the development environment-variable path is deprecated because it would expose the token in instance metadata.

5.4 Verification

The trust-minimisation claim is checked by construction rather than asserted. Because the two data owners never peer with each other and the immutability filter freezes each private input on first receipt, neither owner can observe or alter the other’s secret; both meet only inside the enclave. The trust rests on independent verification of the Google-signed token. The attestation record (secure-boot, debug-disabled, version-matched, with GCE `project_id/zone/instance_id` in the claims) plus the launcher logs together evidence that the released code ran in a genuine Confidential Space and that no private benchmark or model left it.

5.5 Limitations

The attack vectors due to inference code using side channels to reveal benchmarks are out of scope for this paper, and will be addressed in a future version.

5.6 Code

The code is available publicly in Github: <https://github.com/khoaguin/blindfold> and the README.md contains instructions to reproduce the demo.

6 Conclusion & Future Work

We presented Blindfold: a blind evaluation harness and a bilingual local-harms benchmark that together measured the English→Vietnamese safety gap across four models inside

a sealed enclave, so that a lab, a safety org, and an auditor who do not trust each other can run one evaluation and have only a score come out. The headline finding — that the Vietnamese-specialized model is the least safe in Vietnamese — is exactly the blind spot an English-only audit cannot see.

We also demonstrated that **our real distributed deployment** (Section 5) already runs the same audit with a genuine TEE on GCP Confidential Space and remote attestation, turning asserted trust-minimization into a verified one. In future work, we can **expand the benchmark** and the model set, and / or involve real AI labs, Safety orgs, independent researchers in evaluating bigger, production grade models on real eval datasets.

References

- [1] Yue Deng et al. “Multilingual Jailbreak Challenges in Large Language Models”. In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. 2024. arXiv: 2310.06474 [cs.CL]. URL: <https://arxiv.org/abs/2310.06474>.
- [2] Zheng-Xin Yong, Cristina Menghini, and Stephen H. Bach. *Low-Resource Languages Jailbreak GPT-4*. NeurIPS 2023 Workshop on Socially Responsible Language Modelling Research (SoLaR). 2023. arXiv: 2310.02446 [cs.CL]. URL: <https://arxiv.org/abs/2310.02446>.
- [3] DAMO-NLP-SG. *MultiJail: A Multilingual Jailbreak Benchmark*. Dataset, MIT License. MIT License. 2023. arXiv: 2310.06474 [cs.CL]. URL: <https://github.com/DAMO-NLP-SG/multilingual-safety-for-LLMs>.
- [4] Cohere Labs. *The Aya Red-teaming Dataset*. 2024. arXiv: 2406.18682 [cs.CL]. URL: <https://arxiv.org/abs/2406.18682>.
- [5] OpenMined. *syft-client*. <https://github.com/OpenMined/syft-client>. 2024. URL: <https://github.com/OpenMined/syft-client>.
- [6] Vietnam Authority of Information Security (AIS) and Vietnam Ministry of Health. *24 Forms of Online Fraud; Ministry of Health Misinformation Warnings*. Government advisory catalogues. 2023.